EVENTIFY

Documento técnico

SC-502 Ambiente Web Cliente-Servidor
Código de referencia: 0.1.0-dev.52

Estudiante: Kendall Gamboa Arce
Docente: Rita Robles Loaiza

Contenido

1. Descripción y resultado esperado 3

2. Tecnologías y arquitectura 3

3. Sitemap y navegación por roles 4

4. Datos y relaciones 4

5. Reglas transaccionales y decisiones 5

6. Requisitos e historias de aceptación 6

7. Seguridad y errores 10

8. AJAX y API externa 11

9. Instalación, despliegue y datos 11

10. Pruebas y validación 12

11. Conclusiones 13

12. Referencias 13

1. Descripción y resultado esperado

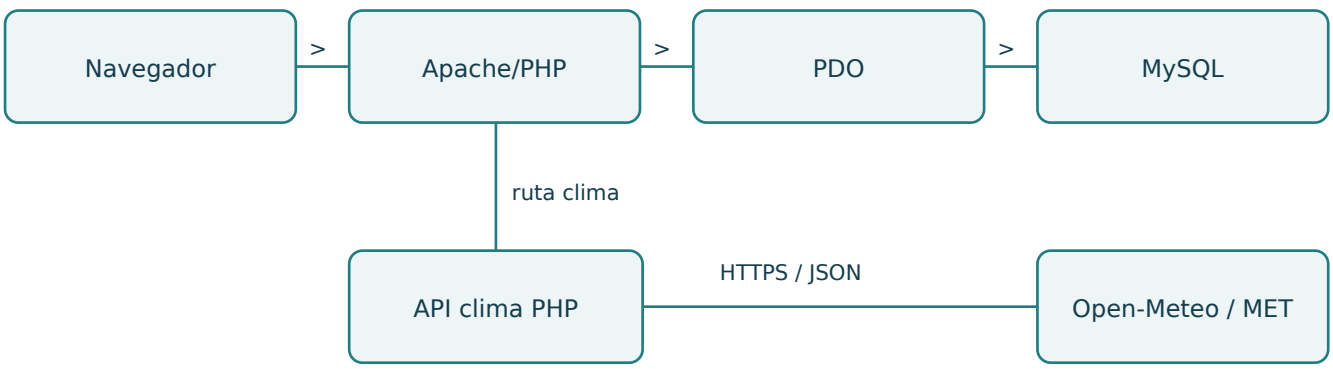
EVENTIFY es una aplicación web para descubrir eventos, reservar entradas y consultar tickets con QR. Un usuario se registra, inicia sesión, filtra eventos y confirma una cantidad de entradas. El administrador gestiona categorías, eventos, reservas y métricas. El pronóstico meteorológico ayuda a planificar eventos próximos; su fallo no debe bloquear reservas.

La aplicación funciona en un entorno local de Windows y en un despliegue web público. Su alcance no incluye cobros reales, aplicación móvil nativa ni control de acceso mediante escaneo.

El alcance corresponde a la suficiencia especial del curso SC-502. La instalación local se facilita mediante un asistente y dos accesos de escritorio para iniciar y detener la aplicación.

2. Tecnologías y arquitectura

Capa	Tecnología y función
Interfaz	HTML5, CSS3 y JavaScript; vistas PHP con escape de contenido.
Interacción	jQuery 4.0.0 local; catálogo e historial asincrónicos, sin CDN obligatorio.
Servidor	PHP 8.4 y Apache; controladores HTTP, servicios de dominio y repositorios PDO.
Datos	MySQL 8.4 Community/InnoDB, restricciones, índices y migraciones versionadas.
Identidad	Sesiones PHP, hashes de contraseña, CSRF y roles comprobados en servidor.
QR	BaconQrCode 3.1.1 y dasprid/enum 1.0.7; generación SVG local con composer.lock.
Entorno	Docker Desktop y Docker Compose; asistente CMD/PowerShell y volúmenes persistentes.
API externa	Open-Meteo y respaldo MET Norway por HTTPS, destinos fijos, timeout, caché y límite global.
Desarrollo	Git y GitHub para conservar el historial; pruebas automatizadas de reglas e integración.



public/index.php es el punto de entrada y decide ruta/método. app/Http coordina respuestas y autorización; app/Domain concentra reglas y transacciones; app/Infrastructure realiza acceso a MySQL/API y migraciones; app/Views renderiza; app/Security administra identidad; public/assets contiene estilos, JavaScript, bibliotecas e imágenes locales. La raíz pública es public, no el repositorio completo.

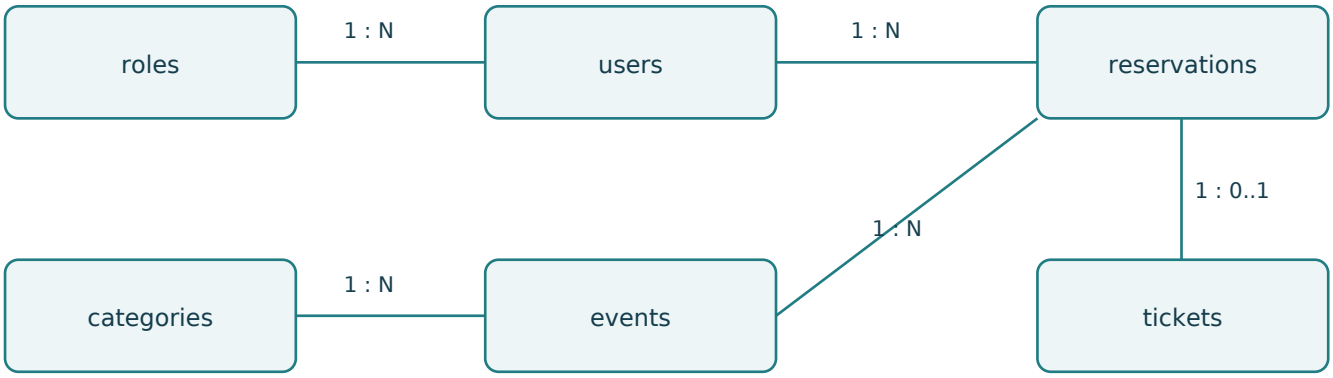
PHP y MySQL se ejecutan dentro de contenedores. Docker reproduce el entorno local sin instalar esos componentes directamente en Windows. Las herramientas de pruebas y generación de documentos son independientes de la ejecución de la aplicación.

3. Sitemap y navegación por roles

Área	Rutas principales y acceso
Pública	/, /eventos, /eventos/{slug}, /registro, /ingresar. Los borradores no son públicos.
Cuenta	GET/POST /cuenta: consultar perfil y editar nombre con sesión/CSRF; correo y rol conservados. POST /salir destruye sesión. Login redirige al inicio.
Usuario	/experiencias, /tickets/{code}, /tickets/{code}/qr.svg; propiedad verificada en MySQL.
Administrador	/admin, /admin/categorias, /admin/eventos, /admin/reservas, /admin/usuarios y detalles/formularios.
AJAX	/api/eventos, /api/experiencias, /api/reservas, cancelación y clima.
Disponibilidad	/salud informa ok/unavailable; no expone claves ni consultas.

La navegación visual no otorga permisos. Una URL de administración exige ADMINISTRADOR; una URL de ticket exige su propietario. Un ticket ajeno responde 404 sin revelar contenido. El registro público asigna USUARIO aunque el navegador envíe otro rol.

4. Datos y relaciones



Una categoría agrupa muchos eventos; un usuario realiza muchas reservas; un evento recibe muchas reservas; una reserva tiene como máximo un ticket. Las claves foráneas restringen eliminación y conservan historia. Las diez tablas incluyen seis entidades de negocio y cuatro técnicas. La migración

003 añade weather_budget para limitar las consultas a los proveedores del clima.

Diccionario y reglas

Tabla	Campos y reglas relevantes
roles	id TINYINT; code ASCII único y nombre. Roles ADMINISTRADOR/USUARIO creados en migración.
users	id BIGINT; role_id; full_name 150; email 254 único sin distinguir mayúsculas; password_hash 255; is_active; fechas de creación/actualización.
categories	id BIGINT; nombre 120 y slug ASCII 120 únicos; indicador activo y fechas.
events	id, categoría; slug ASCII único 160; nombre 180; descripción TEXT; ruta de imagen 255; inicio DATETIME; ubicación 255; coordenadas DECIMAL(9,6) opcionales con límites; capacidad INT positiva; precio DECIMAL(12,2) no negativo; estado y fechas.
reservations	id, usuario/evento; código ASCII único de 32 caracteres; cantidad INT positiva; precio unitario histórico DECIMAL(12,2); total DECIMAL(22,2), igual a precio × cantidad; estado; fecha de cancelación y auditoría temporal.
tickets	id; reserva única; código ASCII único de 64 caracteres; creación. Un ticket representa la reserva completa y su cantidad; QR SVG generado localmente.
schema_migrations	Versión, SHA-256, estado EN_PROCESO/APLICADA, número de sentencias ejecutadas, inicio/finalización.
auth_limits	Migración 002: clave HMAC de acción/IP, inicio de ventana y contador; protección persistente de intentos.
demo_seed_runs	Identificador de demostración aplicada y fecha; evita sembrar de nuevo.
weather_budget	Migración 003: scope global, ventanas hora/minuto y contadores con máximos 30/h y 10/min; persistencia/transacción PDO preparadas.

El esquema exacto está en database/migrations/001_initial_schema.php, 002_auth_limits.php y 003_weather_budget.php. Fechas en UTC, presentación en America/Costa_Rica. Precio histórico unit_price y total se guardan en DECIMAL; el servidor calcula precio por cantidad. active_event_id es columna generada: el índice único impide dos reservas activas por usuario/evento y permite conservar cancelaciones.

No existe una fila de cupos disponibles: se deriva capacidad menos suma de entradas CONFIRMADAS. Reserva y evento tienen estados separados; un evento cancelado invalida el ticket sin borrar su reserva. Imágenes actuales pertenecen a una galería del código; no hay carga arbitraria de archivos.

5. Reglas transaccionales y decisiones

Confirmar reserva inicia una transacción, bloquea la fila del evento FOR UPDATE y revalida estado, fecha, duplicidad, cantidad y disponibilidad. Inserta reserva y ticket antes de COMMIT. Un fallo revierte ambos. La base añade índice único y restricciones, pero el límite de aforo necesita ese bloqueo de servicio.

Cancelar verifica propietario y usa orden de bloqueo evento antes de reserva. Solo se permite a más de 86400 segundos del inicio; exactamente 24 horas se rechaza. Repetir una cancelación ya realizada

no libera cupos dos veces. No se borra historia ni se altera el precio guardado.

Administración conserva slug y precios históricos. Cualquier historial fija la fecha y prohíbe regresar a BORRADOR. No reduce capacidad por debajo de entradas confirmadas. Cancelar un evento impide nuevas reservas, conserva existentes y no equivale a cancelar cada reserva. Finalizar explícitamente exige que haya iniciado. Categoría inactiva conserva referencias.

El dashboard cuenta todos los eventos para el total general. Para los indicadores activos utiliza eventos publicados con inicio futuro y sus reservas confirmadas. La ocupación es entradas reservadas divididas entre capacidad acumulada, multiplicada por 100; si la capacidad es cero muestra cero. Los rankings comparan entradas reservadas y resuelven empates por identificador. Los ingresos potenciales suman los totales históricos de esas reservas y no representan pagos. Las métricas tienen pruebas con datos conocidos, sin inventar ventas reales.

6. Requisitos e historias de aceptación

Los siguientes requisitos corresponden al enunciado de la suficiencia especial. Los criterios indican cómo revisar el comportamiento esperado. Los actores son visitante, usuario y administrador.

RF-01 - Registro de usuarios

- Solicitar nombre completo, correo, contraseña y confirmación; validar cliente y servidor; correo único; contraseñas sin texto plano.

- Aceptación: alta válida persistida; campos inválidos y confirmación distinta rechazados; correo duplicado rechazado también por petición directa; hash en MySQL. El correo se normaliza antes de comprobar duplicados.

RF-02 - Autenticación y cierre

- Ingresar con correo/contraseña y mantener sesión PHP; mensajes comprensibles sin información sensible; destruir sesión al salir.

- Aceptación: acceso válido/incorrecto, persistencia entre peticiones y acceso denegado tras logout. Verificar renovación de identificador de sesión como medida de implementación.

RF-03 - Roles y autorización

- Roles ADMINISTRADOR y USUARIO; autorización en servidor; rechazar URL directa no autorizada. Ocultar botones no es protección.

- Aceptación: visitante/usuario rechazados en endpoints administrativos; administrador accede; registro público no puede elevar su rol enviando campos adicionales.

RF-04 - Catálogo

- Tarjetas u otra vista apropiada con imagen, nombre, categoría, fecha/hora, ubicación, precio, disponibilidad y enlace; distinguir cancelados/finalizados.

- Aceptación: información real de MySQL, estados visibles y caso sin resultados. Borradores accesibles solo al administrador según su estado.

RF-05 - Búsqueda dinámica

- Buscar por nombre sin recarga completa mediante comunicación asincrónica.

- Aceptación: resultados pertinentes, vacío, sin coincidencias y fallo controlado; las respuestas tardías no reemplazan la consulta más reciente.

RF-06 - Filtros combinables

- Nombre y categoría visibles; fecha, disponibilidad y estado en un desplegable accesible. Los criterios se combinan mediante AJAX y también funcionan por GET.

- Aceptación: combinación de los tres criterios, limpiar filtros, rango inválido y consulta sin resultados; validar parámetros en servidor.

RF-07 - Detalle y disponibilidad

- Imagen, nombre, descripción, categoría, fecha/hora, ubicación, precio, capacidad, espacios disponibles; información API cuando corresponda.

- Disponibilidad calculada con reservas vigentes persistidas.

- Aceptación: cantidad de cupos coincide con MySQL; evento inexistente gestionado; disponibilidad actualizada tras reservar/cancelar; fallo API no bloquea el detalle.

RF-08 - Crear reservación

- Usuario autenticado selecciona una o más entradas; total dinámico precio × cantidad; persistir en MySQL.

- Aceptación: reserva válida tiene usuario/evento/cantidad/valor; no autenticado rechazado; cantidades cero/negativas/fraccionarias/inválidas rechazadas. Manipular precio/total en la petición no cambia el cálculo autoritativo del servidor.

RF-09 - Control de cupos

- Comprobar disponibilidad en servidor inmediatamente antes de confirmar; no superar disponible; nunca disponibilidad negativa.

- Aceptación: reservar exactamente los cupos restantes funciona; pedir uno de más falla; dos peticiones simultáneas al último cupo no pueden confirmar ambas. Un fallo a mitad de operación no deja reserva/ticket parcial ni consume cupos.

- Implementación: transacción, bloqueo de la fila del evento y lectura autoritativa de MySQL antes de guardar.

RF-10 - Últimos espacios

- Mostrar automáticamente «¡Últimos espacios disponibles!» cuando disponible ≤ 20 % de capacidad.

- Aceptación: verificar debajo, exactamente en el umbral y por encima; capacidades pequeñas sin errores de redondeo. Mantener la condición matemática del PDF y distinguir agotado en la interfaz.

RF-11 - Reserva activa duplicada

- No mantener dos reservaciones activas independientes del mismo usuario/evento; informar y evitar duplicado.

- Aceptación: segunda petición secuencial y dos peticiones simultáneas no crean dos activas. Nueva reserva tras cancelación permitida solo si el evento admite reservar y existen cupos.

RF-12 - Mis experiencias

- Mostrar evento, fecha, cantidad, total, estado y acceso al ticket de las reservas propias.

- Aceptación: propietario accede; otra cuenta no puede leer reserva/ticket ni por HTML ni por endpoint/URL manipulada; estados e historial correctos.

RF-13 - Cancelar reservación

- Solo si faltan más de 24 horas; cambiar estado sin borrar fila; liberar cupos; con 24 horas o menos impedir e informar motivo.

- Aceptación: 24 h + 1 s permitido, exactamente 24 h y 24 h − 1 s rechazados; comprobar el límite con un reloj controlado en las pruebas. Repetir cancelación no libera dos veces; solo propietario o acción administrativa expresamente definida.

RF-14 - Ticket digital

- Cada reservación confirmada genera ticket con identificador único, evento, usuario, fecha/hora, cantidad, código de reservación y QR generado con biblioteca apropiada.

- Aceptación: registro persistido, código único, QR real renderizable, datos coinciden con reserva; rechazo de acceso ajeno. Un ticket por reserva puede representar varias entradas; no exigir un ticket por entrada.

- El QR contiene el identificador de la reserva y no datos personales. El ticket permanece como historial después de cancelar, pero deja de estar vigente. No se incluye un sistema de escaneo de acceso.

RF-15 - API externa útil

- Mínimo una API; datos integrados de forma útil; manejar indisponibilidad/error. Clima es ejemplo, no obligación.

- Aceptación: respuesta real visible en contexto del evento, error/timeout/cuota/formato inesperado manejados; no inventar datos como si fueran respuesta real.

- Elegir API con condiciones gratuitas compatibles, atribución/licencia documentada, caché razonable y límites temporales/geográficos respetados. Si clima no cubre la fecha, comunicarlo.

RF-16 - Dashboard

- Datos reales: total eventos, eventos activos, reservas activas, entradas reservadas y porcentaje global de ocupación.

- Al menos dos derivados: evento más reservado, categoría más popular, mayor ocupación o ingresos potenciales.

- Aceptación: comparar con conjunto de datos conocido; cantidades de entradas no confundidas con cantidad de reservas; cálculo ponderado y conjunto de eventos publicados cuyo inicio es futuro; sin división por cero. Ingresos potenciales claramente identificados, nunca pagos reales.

RF-17 - Categorías

- Crear, consultar, modificar y eliminar o desactivar; proteger integridad referencial.

- Aceptación: CRUD funcional con validación; categoría utilizada no rompe eventos ni historial al desactivar/eliminar. Decisión inicial: desactivación cuando esté referenciada; no borrar en cascada eventos.

RF-18 - Administración de eventos

- CRUD completo: identificador, nombre, descripción, categoría, imagen, fecha, hora, ubicación, capacidad, precio, estado.

- Estados BORRADOR, PUBLICADO, CANCELADO, FINALIZADO.

- Aceptación: creación, lectura, edición y eliminación segura/desactivación conforme al historial. No borrar eventos reservados para simular cancelación; exponer eliminación real solo donde no destruya historial y esté autorizada.

RF-19 - Validaciones de eventos

- Capacidad > 0; precio ≥ 0; categoría y obligatorios presentes; fechas válidas; no bajar capacidad por debajo de entradas ya reservadas.

- Aceptación: probar cada límite y petición directa; precio cero permitido; rechazo de capacidad insuficiente también frente a una reserva concurrente.

- Reglas adicionales sobre fechas futuras, imagen y cambios en eventos reservados se documentan como decisiones, no como citas textuales del PDF.

RF-20 - Consulta administrativa de reservas

- Administrador consulta todas, ve detalle y filtra por evento, usuario y estado.

- Aceptación: permisos, filtros combinados y estados históricos; consultas consistentes con MySQL, vacío gestionado. No añadir edición arbitraria de reservas por inferencia.

RF-21 - Cancelar eventos

- Cambiar a CANCELADO, bloquear nuevas reservas, conservar existentes y comunicar claramente cancelación.

- Aceptación: evento/reservas históricas permanecen; reservar falla incluso si la pantalla estaba abierta antes; ticket y sección personal muestran cancelación; dashboard usa conjunto coherente.

- Resolver carrera entre cancelación y confirmación de reserva bajo la misma política transaccional.

Historias de usuario y criterios de aceptación

Historia	Actor y objetivo	Criterio de aceptación concreto
HU-01 / RF-01/02	Visitante quiere crear una cuenta e ingresar.	Alta válida guarda hash; correo duplicado y contraseña incorrecta fallan; login regenera sesión.
HU-02 / RF-04/05/06	Usuario quiere encontrar un evento disponible.	Filtros AJAX coherentes; borrador privado; precio cero válido y estado agotado visible.
HU-03 / RF-08/09/11	Usuario quiere reservar entradas sin sobreventa.	Cantidad positiva, total del servidor, último cupo y duplicado concurrente protegidos.
HU-04 / RF-12/14	Usuario quiere consultar su ticket y QR.	Ticket propio visible; ajeno 404; QR local sin información personal en payload.
HU-05 / RF-13	Usuario quiere cancelar a tiempo.	Más de 24 h permitido; límite exacto y después rechazados; historial conservado.
HU-06 / RF-17/18/19/21	Administrador quiere gestionar eventos con seguridad.	CSRF/rol, aforo suficiente, estados válidos y cancelación conservan reservas.
HU-07 / RF-15/16/20	Administrador quiere revisar reservas y métricas.	Filtros combinados y agregados verificables con conjunto conocido; datos de otro rol protegidos.

Requisitos no funcionales

ID	Obligación PDF	Criterio verificable
RNF-01	Usabilidad consistente e intuitiva; retroalimentación.	Formularios etiquetados, mensajes de éxito/error/carga, vacío y confirmaciones pertinentes; navegación por teclado y foco visible como medidas de calidad.
RNF-02	Móvil, tableta y computadora; no solo reducir proporcionalmente.	Verificar 360/390, 768 y 1280/1440 px, zoom y orientación; menú, formularios, tarjetas, tablas y QR utilizables; sin desbordamiento de la página. Estos tamaños cubren móviles, tabletas y escritorio.
RNF-03	Hashing, sesiones, autorización/validación servidor, consultas parametrizadas, escape y secretos por entorno.	Pruebas de permisos/propiedad, inyección SQL/XSS, sesión/logout y exclusión de secretos; protecciones adicionales de sesión y CSRF.
RNF-04	Manejar errores auth/validación/ BD/API/cupos/permisos sin revelar datos internos.	Mensajes seguros y respuestas HTTP coherentes; logs redactados; probar fallos y servicio externo caído.
RNF-05	Evitar consultas y recursos innecesarios; optimizar imágenes; evitar recargas injustificadas.	Revisar consultas repetidas, índices/filtros, tamaño de imágenes, caché API y operaciones AJAX. Las mediciones se registran antes de establecer metas de rendimiento.
RNF-06	Chrome y Firefox actuales.	Flujo real en ambos navegadores; registrar versiones. Herramientas automatizadas no sustituyen una revisión de los navegadores requeridos.
RNF-07	Separar responsabilidades, nombres claros y reutilización; no un único PHP.	Revisión de estructura, funciones compartidas, manejo central de conexión/errores y separación de vistas/reglas/acceso a datos.

7. Seguridad y errores

Las contraseñas se guardan con password_hash y se comprueban con password_verify. Las sesiones usan modo estricto y cookies HttpOnly, SameSite=Lax y Secure en producción. Al ingresar cambia el identificador; al salir se destruye. Tras 30 minutos de inactividad se reinicia la identidad. Los controles HTTPS públicos verificaron estas cookies, la regeneración y el rechazo de una sesión destruida.

Las operaciones de escritura requieren CSRF. El servidor limita métodos HTTP, valida entradas, usa consultas preparadas y escapa el contenido de las vistas. Las consultas de reservas y tickets incluyen el propietario. Un usuario no decide su rol, precio, total ni cupos enviando campos desde el navegador. La web rechaza tokens de otra sesión, tickets y QR ajenos y rutas administrativas sin el rol necesario.

La configuración privada se obtiene mediante variables de entorno y queda fuera del repositorio y de la raíz pública. Los errores no muestran consultas, claves ni trazas al visitante.

La conexión pública a MySQL usa TLS y valida el certificado y la identidad del servidor. Las cuentas públicas de evaluación se preparan expresamente con claves únicas; no se crean administradores con una contraseña universal al iniciar el servidor. La cuenta de la aplicación tiene permisos limitados a su base de datos.

Los contenedores locales usan no-new-privileges para impedir que los procesos adquieran privilegios adicionales al ejecutar archivos. Las dependencias están fijadas por versión. El escaneo de las tres imágenes corregidas del 10 de octubre de 2026 no reportó avisos con la base utilizada en esa fecha. Es un resultado de ese corte: requiere revisiones posteriores y no audita el equipo del usuario ni los proveedores.

8. AJAX y API externa

Operación	Entrada	Respuesta y protección
GET /api/eventos	q, category, from, to, availability, state, page validados.	JSON html/page/message; solo eventos públicos, fragmento escapado.
POST /api/reservas	csrf, event_id, quantity.	201 con message/id/ticket_url/available; sesión, CSRF y transacción.
POST /api/reservas/{id}/cancelar	csrf.	JSON message; propietario y regla de 24 h; errores seguros.
GET /api/experiencias	Cookie de sesión.	JSON html/message del usuario; sin sesión 401.
GET /api/eventos/{slug}/clima	Slug de evento público.	JSON status/message y forecast cuando válido; desconocido 404.

Los errores de negocio usan códigos HTTP acordes (401, 403, 404, 409, 422); métodos no admitidos 405. Un error inesperado puede devolver 500 con texto seguro desde el manejador global, por lo que el cliente debe tolerar respuesta no JSON. No es una API de pagos.

jQuery aporta dos funciones significativas: catálogo con debounce/aborto/control de secuencia e historial actualizado tras cancelar. AJAX tiene más de tres operaciones. El total visual orienta al usuario; el precio definitivo lo calcula PHP/MySQL.

La confirmación cliente tiene espera máxima de 15 s incluyendo JSON. Bloquea doble envío; ante timeout/red/5xx o respuesta inválida dirige a consultar experiencias y evita reintento inmediato desde el formulario. Abortar la espera no cancela una reserva ya confirmada en MySQL. El recorrido de Chrome comprobó el aviso y la navegación al ticket.

Open-Meteo se consulta desde PHP por HTTPS, coordenadas validadas y destino fijo. Lectura acotada a 3 s y 128 KiB, sin seguir redirects; unidades/rangos/fecha revisados. Pronóstico diario solo para evento público futuro dentro de 15 días, con atribución. Caché positiva 1 h, negativa 10 min y resultado anterior hasta 6 h. El fallo permite continuar con la reserva.

La aplicación mantiene una cuota global MySQL de 30 intentos/h y 10/min, consumida antes del HTTP externo bajo lock de fila. Los errores externos cuentan; falla de presupuesto omite la llamada. La caché puede perderse en alojamiento efímero, pero la cuota MySQL debe conservarse. Reglas/callback, migración, límites en MySQL y persistencia/restauración comprobados. Dos carreras reales del presupuesto fueron comprobadas en la integración MySQL de 219 controles.

9. Instalación, despliegue y datos

El despliegue local utiliza Docker Desktop y Docker Compose en Windows. El asistente CMD/PowerShell comprueba las dependencias, configura la aplicación, espera la disponibilidad de MySQL y aplica las migraciones. Los accesos de inicio y detención controlan los servicios sin eliminar los datos persistentes.

Una instalación vacía recibe ocho eventos de ejemplo, cinco categorías y cuentas de administrador y usuario con contraseñas únicas. Los accesos se guardan en storage/example-credentials.json de la carpeta instalada. Si ya existen datos, el arranque los conserva y no reemplaza cuentas ni eventos. Los ejemplos no representan cobros ni actividades contratadas.

Los volúmenes mysql_data y app_storage conservan datos al detener o recrear contenedores. La copia del código no contiene las reservas de una instalación existente: para trasladarlas hay que realizar un respaldo de MySQL y de los archivos. No se deben borrar los volúmenes como parte del uso normal.

La aplicación pública está en https://eventify-academico.onrender.com. Usa PHP/Apache en Render y MySQL en Aiven. La copia local es independiente: no comparte automáticamente cuentas, reservas ni contraseñas con la web pública y no necesita cuentas en esos proveedores. Los planes gratuitos pueden suspender servicios por inactividad, lo que afecta su disponibilidad y el tiempo de la primera respuesta.

Git conserva el código y las versiones, pero no reemplaza un respaldo de la base de datos. Para volver a una versión anterior hay que revisar primero su compatibilidad con las migraciones y los datos existentes.

10. Pruebas y validación

Se combinan pruebas de reglas de negocio, integración PHP/MySQL, peticiones HTTP y recorridos de navegador. Se revisan autenticación, permisos, propiedad del ticket, CSRF, cantidades inválidas, precio calculado en servidor y los límites de cancelación. Las pruebas de concurrencia envían solicitudes simultáneas para revisar el último cupo y las reservas duplicadas.

Área	Resultado documentado	Alcance
Integración de la aplicación dev.46	290 comprobaciones MySQL/HTTP aprobadas.	Incluye catálogo inicial y conservación de datos; no demuestra instalación en Windows.
Interfaz dev.45	173 comprobaciones en Firefox, 22 pantallas y seis tamaños/orientaciones.	Resultado de esa versión, con recorridos dirigidos posteriores del catálogo dev.46.
Clima	58 comprobaciones de proveedores y 44 del presupuesto de consultas.	Éxito real y fallos controlados documentados; el servicio externo puede no estar disponible.
Persistencia	Reinicios y recreación conservan tablas, reservas, tickets y archivos en los ensayos registrados.	Los datos se conservaron en los destinos probados.
Instalador Windows dev.52	68 controles del asistente y ocho de espera aprobados en PowerShell 5.1.	Operaciones de instalación simuladas en esos controles. El usuario confirmó después la reinstalación real de Eventify con Docker existente.

El usuario confirmó el funcionamiento del instalador dev.52 después de eliminar los contenedores, los volúmenes y las imágenes anteriores de Eventify, conservando Docker Desktop y WSL. También confirmó los accesos de inicio y detención. Este resultado corresponde a una máquina virtual con 4 GiB; no demuestra compatibilidad con todos los equipos ni repite la instalación inicial de Docker.

11. Conclusiones

EVENTIFY centraliza la consulta de eventos, las reservas y los tickets. La separación entre interfaz, reglas de negocio y acceso a datos permite mantener las validaciones en el servidor. Las transacciones protegen los cupos y el precio histórico permite conservar el valor de una reserva aunque cambie el evento.

AJAX evita recargas completas en las búsquedas y en las operaciones de reserva. El clima aporta información útil sin convertirse en una dependencia para reservar. Docker facilita la instalación de PHP y MySQL en un entorno reproducible, con datos persistentes.

12. Referencias

- Enunciado SC-502 Suficiencia Especial: EVENTIFY, secciones 5 a 13.
- Programa del curso SC-502 Ambiente Web Cliente-Servidor.
- PHP: https://www.php.net/manual/es/
- MySQL: https://dev.mysql.com/doc/refman/8.4/en/
- jQuery: https://api.jquery.com/
- Docker: https://docs.docker.com/
- Open-Meteo: https://open-meteo.com/en/docs
- MET Norway: https://api.met.no/doc/Locationforecast/2.0/documentation
- BaconQrCode: https://github.com/Bacon/BaconQrCode